

FIELD GUIDE

티키타카 개발 노트

기본기, 실전, Agent, 지식 자산화를 하나의 개발 루프로 만드는 법

학습 50 : 툴링 50

강한 개발자 + 강한 Agent = 더 강한 팀

핵심 AI가 강력해질수록 판단, 검증, 설명을 맡는 개발자의 기본기가 더 중요해진다. Agent는 대행자가 아니라 학습 파트너이자 리뷰어다.

원문 요약 및 실무 보완판

원문: OKKY 커뮤니티 · 작성자 copilot2 · 정리일 2026-08-31

1. 왜 이 방식이 필요한가

AI로 빠르게 기능을 만드는 것 자체는 어렵지 않다. 문제는 사람이 결과를 이해하지 못한 채 속도만 높일 때 발생한다. 설계 이유가 사라지고, 검증되지 않은 코드가 쌓이며, 장애가 났을 때 원인을 설명할 수 없게 된다.

메시지 빠른 생성은 출발점일 뿐이다. 이해, 작은 단위의 구현, 인간의 검증, 기록과 재사용까지 연결되어야 개발 역량이 남는다.

이 문서는 원문의 철학을 유지하면서 실제 팀과 프로젝트에서 반복 가능한 운영 방식으로 확장했다. 원문 요약은 본문 앞부분에, 추가 해설과 실행 도구는 '보완 정리'로 구분했다.

2. 핵심 공식: 학습 50 + 툴링 50



원문 인포그래픽: 기본기와 도구 활용을 5:5로 균형 있게 반복

학습은 기술 원리, 아키텍처, 코드 독해, 문제 해결 능력을 키우는 일이다. 툴링은 Agent, MCP, 자동화, 코드 생성, 테스트, 문서화를 활용해 학습과 실행의 속도를 높이는 일이다. 어느 한쪽만 강하면 지속 가능한 생산성이 나오기 어렵다.

3. 원문이 제안하는 다섯 가지 원칙

1. 기본기부터 쌓는다

프론트엔드, 백엔드, DB, API, 아키텍처, 테스트, 운영의 세부 사항을 모두 암기할 필요는 없다. 다만 전체 시스템이 어떻게 연결되는지는 설명할 수 있어야 한다.

바로 해보기 완료한 코드 한 덩어리를 선택해 입력, 처리, 저장, 출력, 실패 경로를 자신의 말로 설명한다.

2. Agent와 같이 공부한다

Agent에게 개념을 질문하고, 코드 리뷰를 요청하고, 대안을 비교한다. 중요한 것은 답을 받는 데서 끝내지 않고 배운 내용을 다시 정리하는 것이다.

바로 해보기 질문할 때 정답뿐 아니라 판단 기준, 대안, 실패 조건, 검증 방법까지 요청한다.

3. 큰 기능을 한 번에 말하지 않는다

목표를 작은 구현 단위로 나누고 각 단위마다 확인한다. 범위가 작을수록 사람도 이해하기 쉽고 Agent도 맥락을 정확히 잡는다.

바로 해보기 한 번의 작업은 한 가지 동작과 하나의 완료 조건을 갖도록 쪼갬다.

4. 잘 만든 것은 팀의 자산으로 만든다

코드, 컨벤션, API 패턴, DB 설계, 디버깅 방법, 장애 원인, 실패한 시도와 기술 부채를 재사용 가능한 형태로 남긴다.

바로 해보기 문서에는 결과뿐 아니라 왜 그렇게 결정했는지와 언제 다시 쓰면 되는지를 기록한다.

5. 쌓은 지식을 다시 Agent에게 준다

팀의 코딩 방식, 설계 이유, 과거 경험, 기술 선택과 문제 해결 방법을 Agent가 참조할 수 있게 만든다.

바로 해보기 노트를 검색 가능한 작은 단위로 유지하고, 작업 시작 시 관련 문서 묶음을 컨텍스트로 제공한다.

보완 정리

4. 실행 루프: 공부에서 재사용까지

원문의 흐름을 실무 절차로 바꾸면 다음 일곱 단계가 된다. 각 단계는 별도의 산출물을 남기며, 다음 작업의 입력으로 이어진다.

1. 목표 정의: 이번 작업이 해결할 문제와 완료 조건을 한 문장으로 적는다.
2. 기본기 확인: 관련 개념, 데이터 흐름, 실패 가능성을 먼저 훑는다.
3. 작은 단위 설계: API, 도메인, UI, 테스트를 한꺼번에 묶지 말고 검증 가능한 조각으로 나눈다.
4. Agent와 구현: 코드 생성과 함께 대안, 트레이드오프, 예상 부작용을 질문한다.
5. 사람이 검증: 코드를 읽고 실행하며 테스트, 로그, API 계약, 화면 동작을 확인한다.
6. 노트로 정리: 결정 이유, 검증 증거, 실패한 방법, 재사용 조건을 남긴다.
7. 컨텍스트 재공급: 다음 작업에서 관련 노트를 Agent에게 제공하고 결과를 다시 갱신한다.

루프의 완료 조건 코드가 동작하는 것만으로 끝내지 않는다. 사람이 설명할 수 있고, 검증 증거가 남아 있으며, 다음 사람이 다시 찾을 수 있어야 완료다.

5. 작은 작업으로 나누는 기준

작업 분해의 목적은 Agent를 통제하는 데만 있지 않다. 사람이 각 변경의 인과관계를 이해하고 실패 지점을 빠르게 좁히기 위해서다.

- 완료 조건이 하나인가: '검색된다', '저장된다', '권한이 거부된다'처럼 관찰 가능한 결과가 하나여야 한다.
- 변경 범위가 설명 가능한가: 무엇을 바꾸고 무엇을 바꾸지 않는지 명확해야 한다.
- 독립 검증이 가능한가: 테스트, API 호출, 로그, 화면 중 하나로 바로 확인할 수 있어야 한다.
- 되돌리기 쉬운가: 실패해도 영향이 제한되고 이전 상태를 복구하기 쉬워야 한다.
- 지식이 남는가: 구현 뒤에 배운 점과 결정 이유를 짧게라도 기록할 수 있어야 한다.

6. 실전 예시: MES LOT 상태 필터

'LOT 상태 필터를 전부 만들어줘'라고 요청하는 대신 다음처럼 나누면 프론트와 백엔드의 버전 어긋남까지 단계적으로 확인할 수 있다.

1. 계약 먼저: status 쿼리의 허용 값, 기본값, 잘못된 값의 응답을 정의한다.
2. 백엔드 구현: 컨트롤러 파라미터, 조회 조건, 단위 또는 통합 테스트를 추가한다.
3. API 검증: OpenAPI 문서와 실제 요청 응답에서 파라미터가 노출되고 적용되는지 확인한다.
4. 프론트 구현: 필터 UI, URL 상태, API 호출 값을 연결한다.
5. 화면 검증: 선택값, 새로고침 유지, 빈 결과, 오류 상태를 확인한다.
6. 노트 작성: 계약, 구현 위치, 테스트 명령, 버전 순서와 실패 사례를 남긴다.

Agent에게 물을 질문 '이 구현에서 조용히 실패할 수 있는 지점은 무엇인가?', 'API 계약과 화면 상태가 어긋났음을 어떻게 검출할 수 있는가?', '테스트를 가장 작은 단위로 나누면 무엇인가?'

보완 정리

7. 지식 자산화 노트의 최소 구성

노트는 길이보다 재사용성이 중요하다. 아래 항목을 한 화면에서 찾을 수 있으면 사람과 Agent 모두 다시 쓰기 쉽다.

항목	반드시 남길 내용
문제	무엇이 필요했고 어떤 증상이 있었는가
맥락	관련 모듈, 데이터 흐름, 제약 조건, 버전
결정	선택한 방법과 선택 이유
대안	검토했지만 쓰지 않은 방법과 이유
구현	핵심 파일, API, 스키마, 코드 패턴
검증	실행한 테스트, 명령, 화면 확인, 로그
실패 기록	틀린 가정, 재현 조건, 해결 단서
재사용 조건	언제 다시 적용할 수 있고 언제 적용하면 안 되는가
태그	도메인, 기술, 장애 유형, 관련 기능

노트 템플릿: 결과뿐 아니라 맥락, 결정, 검증, 실패, 재사용 조건까지 기록

8. Agent 컨텍스트 팩

Agent가 팀의 지식을 활용하려면 문서를 단순히 많이 모으는 것보다, 작업에 필요한 묶음을 빠르게 꺼낼 수 있어야 한다.

- 프로젝트 규칙: 디렉터리 구조, 의존성 방향, 명명 규칙, 보안 금지사항
- 아키텍처 지도: 주요 모듈, 데이터 흐름, 외부 연동, 책임 경계

- 대표 패턴: 인증, 오류 처리, API 응답, DB 변경, 테스트 작성 예시
- 결정 기록: 기술 선택과 트레이드오프, 바꾸기 어려운 제약
- 실패 지식: 반복되는 장애, 재현 절차, 확인할 로그와 지표
- 현재 상태: 진행 중 작업, 알려진 기술 부채, 다음 검증 포인트

운영 원칙 모든 문서를 한꺼번에 넣지 않는다. 현재 작업과 관련된 최소 문서만 선택하고, 오래된 정보에는 버전과 유효 기간을 표시한다.

9. 위험 신호와 대응

위험 신호	즉시 할 대응
코드는 많은데 설명할 수 없다	변경을 멈추고 데이터 흐름과 실패 경로를 직접 설명한다.
한 프롬프트가 여러 계층을 동시에 바꾼다	API, 도메인, 저장소, UI, 검증을 별도 작업으로 분리한다.
테스트가 Agent가 만든 코드만 따라간다	요구사항과 실패 조건에서 독립적으로 테스트 케이스를 다시 만든다.
노트가 결과 복사본에 그친다	결정 이유, 대안, 검증 증거, 재사용 조건을 추가한다.
과거 문서가 현재 코드와 어긋난다	문서에 기준 버전과 마지막 검증일을 적고 변경 시 함께 갱신한다.
Agent가 자신 있게 틀린다	근거 파일과 실행 결과를 요구하고, 불확실한 내용은 가정으로 표시한다.

보완 정리

10. 30일 도입 계획

1주차 · 한 기능을 작게

- 진행 중인 기능 하나를 선택한다.
- 작업을 3-5개 검증 가능한 단위로 나눈다.
- 각 단위에 완료 조건과 검증 방법을 적는다.

2주차 · 노트 형식을 고정

- 최소 노트 템플릿을 팀 저장소에 만든다.
- 완료한 작업 두 건을 템플릿으로 정리한다.
- 실패한 시도 한 건을 반드시 포함한다.

3주차 · Agent 컨텍스트 연결

- 프로젝트 규칙과 대표 패턴을 컨텍스트 팩으로 묶는다.
- 작업 시작 시 관련 노트를 Agent에게 제공한다.
- 답변이 실제 규칙을 반영했는지 확인한다.

4주차 · 측정하고 다듬기

- 재작업, 검증 실패, 문서 재사용 사례를 모은다.
- 쓸모 없는 항목은 줄이고 빠진 검증 항목은 추가한다.
- 다음 달에 자동화할 한 가지를 고른다.

11. 측정 지표

Agent Engineering의 성과를 코드 생산량만으로 보면 잘못된 최적화를 하기 쉽다. 속도와 함께 이해, 검증, 재사용을 측정한다.

- **이해 가능성:** 변경자가 데이터 흐름과 설계 이유를 설명할 수 있는 비율
- **검증 통과율:** 첫 구현 후 독립 검증에서 통과한 작업의 비율
- **작은 배치 시간:** 하나의 완료 조건을 구현하고 검증하는 데 걸린 시간
- **지식 재사용률:** 새 작업에서 기존 노트나 패턴을 실제로 참조한 비율
- **결함 유출:** 검증 단계에서 잡히지 않고 다음 환경으로 넘어간 결함 수
- **문서 신선도:** 현재 코드 기준으로 확인된 핵심 문서의 비율

12. 제품 로드맵에 적용하기

원문에서 소개한 현재 기능은 MES LOT 관리 학습 노트와 Agent 기반 계층형 노트 관리다. 향후 기능도 단순 메뉴 추가가 아니라 지식 루프의 어느 단계를 강화하는지 기준으로 설계할 수 있다.

- **MCP를 통한 Agent 연동:** 필요한 노트를 작업 시점에 검색하고 공급
- **디자인 노트 관리:** UI 결정과 근거, 재사용 가능한 패턴 축적
- **프롬프트 라이브러리:** 좋았던 질문 구조와 검증 프롬프트 재사용
- **개발 패턴·코딩 컨벤션:** 팀의 기본 구현 방식을 명시
- **디버깅 지식 베이스:** 증상, 원인, 로그, 해결 순서를 검색 가능하게 저장
- **기술 부채 관리:** 임시 결정과 갚아야 할 조건을 추적
- **팀 지식 직접 참조:** Agent가 현재 규칙과 과거 결정을 근거로 답하도록 연결

13. Git 및 릴리즈 링크

작업 폴더의 실제 Git 원격 주소를 기준으로 정리했다. Tauri 학습 앱의 배포·업데이트 확인에는 pkt-study-fullstack 저장소와 릴리즈 페이지를 사용한다.

Tauri 앱 소스: [dota-pilot1/pkt-study-fullstack](https://github.com/dota-pilot1/pkt-study-fullstack)

최신 릴리즈: [pkt-study-fullstack/releases/latest](https://github.com/pkt-study-fullstack/releases/latest)

전체 릴리즈: [pkt-study-fullstack/releases](https://github.com/pkt-study-fullstack/releases)

이슈 및 개선 제안: [pkt-study-fullstack/issues](https://github.com/pkt-study-fullstack/issues)

상위 샘플 프로젝트: [dota-pilot1/sample-pkt-project](https://github.com/dota-pilot1/sample-pkt-project)

릴리즈 확인 설치 파일이나 업데이트를 받을 때는 '최신 릴리즈'에서 버전, 게시일, 첨부 자산을 확인한다.
실험용 design-playbook-tauri 저장소는 이 문서의 릴리즈 대상으로 사용하지 않는다.

학습 50 + 툴링 50 = Agent Engineering

사람이 공부하고, Agent와 구현하고, 사람이 검증하고, 결과를 기록하고, 팀과 Agent가 다시 활용한다.

이 순환을 꾸준히 돌리는 것이 티키타카 개발 노트의 핵심이다.

출처 및 편집 원칙

이 문서는 아래 원문의 핵심 주장과 공개된 시각 자료를 바탕으로 요약·재구성했으며, 실행 루프, 작업 분해 기준, MES 예시, 노트 템플릿, 위험 대응, 30일 계획, 측정 지표는 이해와 적용을 돕기 위해 보완한 내용이다.

원문: [티키타카 개발 노트 | OKKY 커뮤니티](#)

릴리즈: [pkt-study-fullstack 최신 릴리즈](https://github.com/pkt-study-fullstack/releases/latest)